01-3 logistic回归 数据准备

2026/4/4

写在前面：本节课偏操作，直接看视频就好😉

Tidyverse版本数据分析 - 续

- UP主讲了`R`语言中如何定义函数。
  - Q：什么是函数？A：把一定功能“打包”（也就是计算机术语中的“封装”）起来的代码块。这个代码块可以复用，这样，你在每次用到这个功能的时候就不用重复写一大坨代码了。你已经见过了很多`R`语言中、`tidyverse`中内置的函数了，当然，函数是可以自定义的！
  - 例如，下面就是一个简单的功能封装的例子：

    ```
    dummy_data <- ... # dummy_data是一个演示用的向量
    dummy_data_mean = sum(dummy_data) / n() # 求dummy_data的均值
    mean(dummy_data) # 上面和下面是等效的
    ```

    在这个例子中，我们把求和这个功能用一个简单的函数调用`mean()`代替了，实际上，一个函数可能有几百甚至上千行的代码实现！但是你在调用时，一般只要写几行就可以搞定。
  - 函数定义的基本格式如下：

    ```
    <函数名> <- function(<参数1>, <参数2>, ...) {
        <执行代码>
        return(<返回结果>)  # 可省略
        }
    ```

    - 代码块中，`<函数名>`是你给这个函数起的名字，比如`my_fun`、`count_na`、`plot_box`；
    - `function`是一个**保留字**，这是一个固定用法，你通过写这几个字母来表示你在**声明**一个函数；
    - `<参数1>, ...`是你这个函数需要传入的参数，比如，你在求和时，`mean(data)`，括号内的`data`就是你传入的一个参数；你在调用函数时，传入参数的位置和顺序需要和你函数定义的一致；
    - `<执行代码>`是你这个函数实际执行的逻辑，也就是一大块代码；
    - `return`也是一个**保留字**，它表明你这个函数要**返回**什么结果。还以是`mean()`为例，这个函数返回的结果自然就是计算出来的平均值。
  - 下面是一个函数定义的实际的例子：

```
# 定义：算平方
square <- function(x) {
result <- x * x
return(result)
}

# 使用
square(5)  # 输出 25
```

- 注意！`tidyverse`中函数定义有一定特殊之处！请看本文档的最后！

- UP主又介绍了一个函数，`count()`。这个函数是`dplyr`包里的，它的作用是按某个变量分组，自动统计每组有多少个（频数），它等效于：

```
group_by(变量) %>% summarise(n = n())
```

  - 它的用法是：

```
library(dplyr)

# 按某一列计数
count(数据, 列名)
```

  - 举一个例子，

```
# 模拟数据
df <- data.frame(
性别 = c("男","男","女","女","女"),
分组 = c("治疗","对照","治疗","对照","对照")
)

# 统计男女人数
count(df, 性别)

# 执行结果
  性别 n
1    女 3
2    男 2
```

  - 它有以下几种常见的用法：
    - 单列计数（最常用） - `df %>% count(性别)`
    - 多列交叉计数（行 × 列人数） - `df %>% count(性别, 分组)`，输出为“性别 × 分组”的列联表人数
    - 排序（从多到少） - `df %>% count(分组, sort = TRUE)`
  - 它的常用的场景有：

        - 统计各组人数、统计性别分布、统计疾病分型数量、统计不良反应发生数等等等等
    - 它和 `table()` 的区别在于：
        - `table()`：输出表格样式
        - `count()`：输出 `tibble` 数据框，能直接继续画图 / 分析，更适合 `tidyverse` 流程
- UP主还介绍了一个 `ungroup()` 函数，它的作用是取消数据框的分组属性，避免后续操作被分组影响。
    - 如果你用了以下函数，数据就会变成分组数据：`group_by()`、`count()`、`summarise()` 等；
    - Q：为什么要取消分组？A：因为分组数据会影响后续操作！例如，画图时自动分面 / 分组；计算时按组算，不是整体算；导出表格、排序、变形时出错。
- `tidyverse` 的函数定义与base `R` 有一定不同之处，不同在于 `tidyverse` 中在函数体部分引用传入参数时，需要用 `{{var}}` 的形式，例如：

```
cat_by_outcome = function(var) {
    data_analysis %>% count(TreatmentOutcome, {{var}}) %>%
        group_by(TreatmentOutcome) %>% mutate(prop = n / sum(n)) %>% ungroup()
    }
```

    - 这里，传入的参数 `var` 在函数体里被引用时，需要用 `{{var}}` 的形式。
    - **（以下比较偏原理，可以不看）** 这是 `tidyverse` 独有的 **“数据掩码 + 非标准求值”** 机制，专门为了让你写代码时不用加引号、不用写 `$`。
        - 对于普通 `R` 函数定义，你必须传**列名字符串**才可以正常调用，例如，

```
my_plot <- function(data, col) {
ggplot(data, aes(x = col, y = value)) + geom_point()
}

# 调用必须这样：
my_plot(data, "AGE")  # 要加引号！
```

        - 而对于 `tidyverse` 中的函数定义，你可以直接写列名，不用引号的形式进行调用：

```
my_plot <- function(data, col) {
ggplot(data, aes(x = {{ col }}, y = value)) + geom_point()
}

# 调用时更加方便：
my_plot(data, AGE)  # 直接写 AGE，不用引号！
```

        - 总结，`{{变量}}` 的作用是让函数参数**直接变成数据框里的列名**，是一种**透明传递**的语法。
- UP主定义的函数逻辑还是比较抽象，需要进一步解释一下：
    - 定义的原函数是：

```
cat_by_outcome = function(var) {
    data_analysis %>% count(TreatmentOutcome, {{var}}) %>%
        group_by(TreatmentOutcome) %>% mutate(prop = n / sum(n)) %>% ungroup()
    }
```

- 第一层，`data_analysis %>% count(TreatmentOutcome, {{var}})`，它的作用是按“治疗结局 + 你传入的分类变量”进行交叉计数，得到治疗 有效 / 无效 里，`var` 变量的数量有多少，例如，若 `var` 是性别，则这一步的结果示例如下（**注意，这时 `tibble` 中就有 `n` 这一列了**）：

```
TreatmentOutcome     性别    n
    有效              女     50
    有效              男     30
    无效              女     20
    无效              男     40
```

- 第二层，`group_by(TreatmentOutcome)`，**按治疗结局分组**，让后续计算在每组内部进行。
- 第三层，`mutate(prop = n / sum(n))`，计算每组（**治疗结局**！）内部的构成比，`n` 是**亚组**（治疗结局+ `var` 双重分组后的结果）人数，`sum(n)` 是在**对应治疗结局组**中计算的总人数；`prop` 是新建的一列，语义是比例。

-